

Lab 4 Vibe Coding - Chat History (PES2UG24CS088)

> `<ide_opened_file>`The user opened the file

c:\Users\Aryan\Desktop\24_treasure_hunt\game.py in the IDE. This may or may not be related to the current task.</ide_opened_file>

> **Here is my pygame treasure hunt game (game.py). Before I change anything,** explain how it works: how the world is generated, how GameEngine, Player and the grid interact, how movement and collision work, and how key and chest pickup are handled. Also point out any bugs or edge cases you notice. Don't change any code yet.

- Failed to refresh OAuth token: another Claude Code process is refreshing it or exited mid-refresh. This is usually transient; retry in a minute, and if it persists close other Claude Code processes or sign in again

> **Here is my pygame treasure hunt game (game.py). Before I change anything,** explain how it works: how the world is generated, how GameEngine, Player and the grid interact, how movement and collision work, and how key and chest pickup are handled. Also point out any bugs or edge cases you notice. Don't change any code yet.

Read 1 file (ctrl+o to expand)

- I read through game.py and haven't changed anything. Here's how it works, then the bugs and edge cases I found.

How it works

1. World generation – generate_world() (game.py:9-43 (game.py#L9-L43))

- Grid: The map is a 20×15 list of lists (grid[row][col]). Every cell starts as WALL. Each tile is 40px.
- Rooms: The code makes 8 attempts to place a random room, 3–6 tiles wide and 3–5 tall.
 - The position range keeps a one-tile wall border around the map edge.
 - Each new room is grown by one tile on every side (inflate(2,2)) and checked against the rooms already placed. If it overlaps one, it's thrown away. So rooms always have at least one wall tile between them.
 - Accepted rooms are filled with FLOOR.
- Corridors: Each room is joined to the next one in the list by an L-shaped, one-tile-wide corridor. It runs horizontally along room A's center row to room B's center column, then vertically to room B's center. Both ends are room centers, which are already floor, so every room ends up connected in a chain.
- Items: The chest goes at the center of the last room and the key at the center of the second-to-last room. The player starts in rooms[0].

2. How GameEngine, Player and the grid fit together

- GameEngine owns everything: the pygame window, the clock, the grid, the player and the win/status state.
 - reset() builds a new world and puts the player at the top-left tile of the start room, 6px in from the corner. A 28px sprite at that spot sits exactly in the middle of the tile.
 - run() is a standard loop: handle_events → update → draw → tick(60).
- Player doesn't store the grid. It's only a pixel-space Rect plus a has_key flag.
 - The engine passes the grid into move() each frame. Collision is the only place the player reads the grid.
 - The engine owns pickups and changes the grid there.
- The grid is the single source of truth, for both drawing and collision. Pixel coordinates convert to tile coordinates with // TILE.

3. Movement and collision (game.py:58-73 (game.py#L58-L73))

- Input: `move()` reads the held arrow or WASD keys into `dx` and `dy` ($\pm 3\text{px}$ per frame).
- Axis-separated moves: It calls `_try_move` twice, once for `x` and once for `y`. That's why you slide along a wall when moving diagonally into it instead of stopping dead.
- The collision check: `_try_move` works out the new rect and tests its four corner pixels (using `right-1` and `bottom-1` so the check stays inside the rect). If any corner lands in a WALL tile or outside the grid, the whole step is rejected. Otherwise the new rect is accepted.
- Chests and keys are walkable: Only WALL blocks movement.
- No tunneling: At 3px per frame the player can't skip past a 40px wall.

4. Key and chest pickup (`game.py:112-126` (`game.py#L112-L126`))

- After moving, `update()` finds the tile under the player's center point.
- Key: Stepping on the KEY tile sets `has_key`, turns that tile back into FLOOR (so it disappears) and updates the status text.
- Chest: Stepping on the CHEST tile while holding the key sets `won = True`. After that, `update()` returns early, so the player freezes and the win overlay is drawn.
- Restart: Pressing R at any time calls `reset()`, which creates a new world and a new Player, so `has_key` is cleared too.

Bugs and edge cases

Real bugs

1. Unwinnable maps. Chest and key are only placed if at least 2 rooms were accepted (`game.py:37` (`game.py#L37`)). If 7 of the 8 attempts overlap, you get one room with no key and no chest, but the status still says "Find the KEY, then the CHEST!". Nothing tells the player; they have to guess to press R.
2. Dead fallback code. The first room attempt can never overlap anything, so rooms is never empty and start is never None. The else branch at `game.py:100-101` (`game.py#L100-L101`) is unreachable. If it ever did run, (46, 46) would be inside a wall tile and the player would be stuck.

Gameplay and feel

3. Key can be in the start room. With exactly 2 rooms, `rooms[-2]` is `rooms[0]`, so the key spawns in the room you start in. That's not a crash, but it makes that map trivial.
4. Can't touch walls. When a step would enter a wall, `_try_move` rejects the whole step instead of moving up to the wall. Positions only change in steps of 3 and walls sit on multiples of 40, so the player often stops 1-2px short of a wall.
5. Diagonal movement is faster. Moving diagonally goes about $1.41\times$ faster, since you get 3px on each axis.
6. Opposite keys don't cancel. Holding left and right moves you right; holding up and down moves you down. The later if wins.
7. Corridor entrances need exact alignment. A corridor is 40px wide and the player is 28px , so the player has to be within 12px of lined up to enter one. There's no corner-assist, so turning into a corridor can feel sticky.
8. No feedback at the chest without the key. Walking onto the chest without the key does nothing and the status text doesn't change.
9. Movement depends on frame rate. Speed is measured per frame, not per second, so if the game drops below 60 FPS the player slows down.

Cosmetic or harmless

10. Chained corridors look messy. Rooms are linked in the order they were created, not by distance. Corridors can be long, cut through other rooms or run alongside them, which makes odd merged shapes.
11. Extra frame on quit. After a QUIT event, `update()` and `draw()` still run once more before the loop exits. This is harmless.
12. Redundant bounds checks. The checks at `game.py:71` (`game.py#L71`) and `game.py:118` (`game.py#L118`) can never fail, because the border is always wall. They're fine to keep as safety.

If you want to fix any of these, #1 matters most. The simplest fix is to regenerate the map until it has at least 2 rooms (or 3, which also fixes #3).

> `<ide_opened_file>`The user opened the file

c:\Users\Aryan\Desktop\24_treasure_hunt\game.py in the IDE. This may or may not be related to the current task.</ide_opened_file>

- > **Based on your analysis, fix only the real bugs in game.py with minimal changes.** Don't rewrite the file and don't add any new features yet. Specifically:

If generate_world ends up with fewer than 2 rooms, no key or chest is placed and the level can't be won. Make sure a valid level with both a key and a chest always exists (for example by regenerating until there are at least 2 rooms).

If the player steps on the chest without the key, show a status message like "You need the KEY first!".

After the fix, show me only the changed parts, with a short explanation of each change and why it works.

- Update(game.py)
 - └ Added 5 lines, removed 4 lines
 - 34 grid[cy][bx] = FL00R
 - 35 cy += 1 if by > cy else -1
 - 36
 - 37 - if len(rooms) >= 2:
 - 38 - cr, ck = rooms[-1], rooms[-2]
 - 39 - grid[cr.centery][cr.centerx] = CHEST
 - 40 - grid[ck.centery][ck.centerx] = KEY
 - 37 + if len(rooms) < 2:
 - 38 + return generate_world()
 - 39 + cr, ck = rooms[-1], rooms[-2]
 - 40 + grid[cr.centery][cr.centerx] = CHEST
 - 41 + grid[ck.centery][ck.centerx] = KEY
 - 42
 - 43 start = rooms[0] if rooms else None
 - 44 return grid, start
- Update(game.py)
 - └ Added 6 lines, removed 3 lines
 - 122 self.player.has_key = True
 - 123 self.grid[pr][pc] = FL00R
 - 124 self.status = "Got the key! Find the CHEST!"
 - 125 - elif cell == CHEST and self.player.has_key:
 - 126 - self.won = True
 - 127 - self.status = "Treasure found!"
 - 125 + elif cell == CHEST:
 - 126 + if self.player.has_key:
 - 127 + self.won = True
 - 128 + self.status = "Treasure found!"
 - 129 + else:
 - 130 + self.status = "You need the KEY first!"
 - 131
 - 132 def draw(self):
 - 133 self.screen.fill((30,25,40))

Ran 1 shell command (ctrl+o to expand)

- I fixed both bugs with two small edits in game.py. I generated 2,000 maps with a script and each one had exactly one key and one chest. I didn't launch the game to check the chest message by playing it.

1. Levels can always be won (game.py:37-41 (game.py#L37-L41))

```
if len(rooms) < 2:
    return generate_world()
cr, ck = rooms[-1], rooms[-2]
grid[cr.centery][cr.centerx] = CHEST
grid[ck.centery][ck.centerx] = KEY
```

Before: if len(rooms) >= 2: just skipped placing the key and chest, which left a level you couldn't win.

Now: a map with fewer than two rooms is thrown away and a fresh one is generated. The code below only runs once there are at least two rooms, so the key and chest always go in two different rooms, and the corridors already connect every room.

Getting 7 overlaps in a row is rare, so this almost always takes one or two tries. The function calls itself to retry, but the chance of retrying enough times to hit Python's recursion limit is effectively zero.

2. Message at the chest without the key (game.py:125-130 (game.py#L125-L130))

```
elif cell == CHEST:
    if self.player.has_key:
        self.won = True
        self.status = "Treasure found!"
    else:
        self.status = "You need the KEY first!"
```

Before: the condition was `cell == CHEST` and `self.player.has_key`, so standing on the chest without the key matched nothing and no message appeared.

Now: the check matches any time the player is on the chest, and `has_key` decides between winning and the new message.

The message stays on screen after you step off the chest. It's replaced with "Got the key! Find the CHEST!" when you pick up the key. That seemed fine to me, so I didn't add code to clear it.

> **<ide_opened_file>The user opened the file**

c:\Users\Aryan\Desktop\24_treasure_hunt\game.py in the IDE. This may or may not be related to the current task.</ide_opened_file>

> **Now add Task 1 to game.py: floor traps. Make minimal changes to the existing code and don't rewrite the file.**

Add at least two trap types with different colours, for example spike traps and pit traps. Each type shows its own status message when triggered (e.g. "Spiked! Back to start!" and "Fell in a pit! Back to start!"). Place traps randomly on floor tiles inside rooms only. Keep them out of the start room, off the key and chest tiles, and off corridor tiles, so the level always stays winnable.

When the player steps on a trap, reset the player to the starting position and show the status message. Keep the key state as it is, meaning if the player already has the key, they keep it.

Put the reset logic in one method that I can reuse later for the guard.

Traps should be regenerated when the game restarts with R.

Show me only the changed or new code, with a short explanation of each change.

Read 1 file (ctrl+o to expand)

● Now the edits:

● Update(game.py)

```
└ Added 5 lines
  4  TILE = 40
  5  COLS, ROWS = 20, 15
  6  WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
  7  +SPIKE, PIT = 4, 5
  8  +TRAP_MESSAGES = {
  9  +      SPIKE: "Spiked! Back to start!",
 10  +      PIT: "Fell in a pit! Back to start!",
 11  +}
 12  SPEED = 3
 13
 14  def generate_world():
```

● Update(game.py)

```
└ Added 3 lines
 27                      for rx in range(x, x+w):
```

```

28             grid[ry][rx] = FL00R
29
30 +     corridor = set()
31     for i in range(len(rooms)-1):
32         ax, ay = rooms[i].centerx, rooms[i].centery
33         bx, by = rooms[i+1].centerx, rooms[i+1].centery
34         cx = ax
35         while cx != bx:
36             grid[ay][cx] = FL00R
37 +             corridor.add((cx, ay))
38             cx += 1 if bx > cx else -1
39         cy = ay
40         while cy != by:
41             grid[cy][bx] = FL00R
42 +             corridor.add((bx, cy))
43             cy += 1 if by > cy else -1
44
45     if len(rooms) < 2:

```

- Update(game.py)

- └ Added 28 lines

```

47     cr, ck = rooms[-1], rooms[-2]
48     grid[cr.centery][cr.centerx] = CHEST
49     grid[ck.centery][ck.centerx] = KEY
50 +     place_traps(grid, rooms, corridor)
51
52     start = rooms[0] if rooms else None
53     return grid, start
54
55 +def place_traps(grid, rooms, corridor):
56 +     spots = [(x, y) for room in rooms[1:]]
57 +         for y in range(room.top, room.bottom)
58 +         for x in range(room.left, room.right)
59 +         if grid[y][x] == FL00R and (x, y) not in corridor]
60 +     for _ in range(50):
61 +         traps = random.sample(spots, min(len(spots), random.ran
+rint(3, 6)))
62 +         for x, y in traps:
63 +             grid[y][x] = random.choice((SPIKE, PIT))
64 +             if is_winnable(grid, rooms[0].x, rooms[0].y):
65 +                 return
66 +         for x, y in traps:
67 +             grid[y][x] = FL00R
68 +
69 +def is_winnable(grid, sx, sy):
70 +     seen, stack, found = {(sx, sy)}, [(sx, sy)], set()
71 +     while stack:
72 +         x, y = stack.pop()
73 +         found.add(grid[y][x])
74 +         for nx, ny in ((x+1,y), (x-1,y), (x,y+1), (x,y-1)):
75 +             if (nx, ny) not in seen and grid[ny][nx] in (FL00R,
+ KEY, CHEST):
76 +                 seen.add((nx, ny))
77 +                 stack.append((nx, ny))
78 +     return KEY in found and CHEST in found
79 +
80     COLORS = {
81         WALL: (60,50,70),
82         FL00R: (200,190,170),
83         CHEST: (200,160,30),
84         KEY: (220,220,60),
85 +         SPIKE: (190,60,60),
86 +         PIT: (20,15,10),
87     }
88
89     class Player:

```

- Update(game.py)

- └ Added 1 line

```

136             sy = start.y * TILE + 6

```

```

137         else:
138             sx, sy = TILE+6, TILE+6
139 +         self.start_pos = (sx, sy)
140         self.player = Player(sx, sy)
141         self.won = False
142         self.status = "Find the KEY, then the CHEST!"

```

- Update(game.py)

- └ Added 4 lines

```

141         self.won = False
142         self.status = "Find the KEY, then the CHEST!"
143
144 +     def send_to_start(self, message):
145 +         self.player.rect.topleft = self.start_pos
146 +         self.status = message
147 +
148     def handle_events(self):
149         for event in pygame.event.get():
150             if event.type == pygame.QUIT: return False

```

- Update(game.py)

- └ Added 2 lines

```

169                 self.status = "Treasure found!"
170             else:
171                 self.status = "You need the KEY first!"
172 +         elif cell in TRAP_MESSAGES:
173 +             self.send_to_start(TRAP_MESSAGES[cell])
174
175     def draw(self):
176         self.screen.fill((30,25,40))

```

- The editor reported the file changed on disk, so I'll re-read it to confirm the current state before testing.

Read 1 file (ctrl+o to expand)

- The file contains just my edits. Now a quick headless test of the constraints and the reset behaviour:

Ran 1 shell command (ctrl+o to expand)

- I added spike and pit traps to game.py. I tested it with a headless script but didn't play it in the game window:
 - Placement: I generated 3,000 maps. Every one had traps (about 4.5 on average), none were in the start room, and every map could still be won.
 - Trap trigger: I put a player holding the key on a trap. They went back to the start position, kept the key, and got the right message.

1. Trap tile types and messages (game.py:7-11 (game.py#L7-L11))

```

SPIKE, PIT = 4, 5
TRAP_MESSAGES = {
    SPIKE: "Spiked! Back to start!",
    PIT: "Fell in a pit! Back to start!",
}

```

Traps are new values in the grid, just like KEY and CHEST. The dict also works as the list of trap types: to add a new trap, add one entry here and one colour.

2. Recording corridor tiles (game.py:30-43 (game.py#L30-L43))

```

corridor = set()
...
    grid[ay][cx] = FLOOR
    corridor.add((cx, ay))
...
    grid[cy][bx] = FLOOR
    corridor.add((bx, cy))

```

Once a corridor is carved, it looks like any other floor tile. This set remembers which tiles were corridor so traps can avoid them. That includes

corridor tiles that run through a room.

3. Placing traps (game.py:50 (game.py#L50), game.py:55-67 (game.py#L55-L67))

```
place_traps(grid, rooms, corridor)
...
def place_traps(grid, rooms, corridor):
    spots = [(x, y) for room in rooms[1:]
              for y in range(room.top, room.bottom)
              for x in range(room.left, room.right)
              if grid[y][x] == FLOOR and (x, y) not in corridor]
    for _ in range(50):
        traps = random.sample(spots, min(len(spots), random.randint(3, 6)))
        for x, y in traps:
            grid[y][x] = random.choice((SPIKE, PIT))
            if is_winnable(grid, rooms[0].x, rooms[0].y):
                return
        for x, y in traps:
            grid[y][x] = FLOOR
```

- Which tiles can get a trap: Only room tiles qualify. rooms[1:] skips the start room, == FLOOR skips the key and chest tiles, and the corridor set skips corridor tiles.
- How many traps: Each map gets 3–6 traps, each randomly a spike or a pit.
- Why there's a check: The rules above still aren't quite enough. For example, a full row of traps across a 3-wide room could cut off the key or chest.
- Retry: If the check fails, the traps are removed and the layout is tried again, up to 50 times. If every attempt fails, the map is left with no traps, so it's always winnable. That never happened in my test.
- Restart: Traps are placed inside generate_world(), so pressing R places a new set.

4. Winnability check (game.py:69-78 (game.py#L69-L78))

```
def is_winnable(grid, sx, sy):
    ...
    if (nx, ny) not in seen and grid[ny][nx] in (FLOOR, KEY, CHEST):
    ...
    return KEY in found and CHEST in found
```

This is a flood fill from the start tile that only moves through floor, key and chest tiles. Traps count as blocked. It passes only if both the key and the chest can be reached. The map always has a wall border, so the fill can't step off the grid.

5. Trap colours (game.py:85-86 (game.py#L85-L86))

```
SPIKE: (190,60,60),
PIT: (20,15,10),
draw() already colours tiles with COLORS[cell], so drawing traps needed no other change. Spikes are red and pits are nearly black. The pit is darker than the wall colour, but the two might still be hard to tell apart; I can add a small shape on top of each trap if you want.
```

6. Reusable reset method (game.py:139 (game.py#L139), game.py:144-146 (game.py#L144-L146))

```
self.start_pos = (sx, sy)
...
def send_to_start(self, message):
    self.player.rect.topleft = self.start_pos
    self.status = message
- Where the start is stored: reset() now saves the start position, and send_to_start moves the player there.
- Key is kept: It moves the existing player instead of creating a new Player, so has_key stays as it was.
- For the guard: It can reuse this as self.send_to_start("Caught by the guard!").
```

7. Triggering a trap (game.py:172-173 (game.py#L172-L173))

```
elif cell in TRAP_MESSAGES:
```

```
self.send_to_start(TRAP_MESSAGES[cell])
```

This uses the same check on the player's centre tile as the key and chest. Traps stay on the map after they go off, and the start tile can never be a trap, so the player can't get stuck in a loop of being sent back.

- > **Now add Task 2: an enemy guard. Make minimal changes to the existing code and don't rewrite the file.**

Create a Guard class, separate from Player, that patrols back and forth between two patrol points near the chest, moving a fixed number of pixels each frame. It should stay on floor tiles and never go into walls. Draw the guard in a clearly different colour from the player, traps, key and chest.

If the player touches the guard (rectangle collision), reset the player to the start position using the existing reset method, and show a status message like "Caught by the guard! Back to start!". If the player has the key, they keep it, as with traps.

Guard should be created again when the game restarts with R.

Make sure the guard doesn't sit exactly on the chest tile or on any trap tile.

Show me only the changed or new code, with a short explanation of each change.

- Plan: reserve the room row directly above the chest as the guard's patrol lane when the world is generated. Traps can't be placed on that row, so the lane is always clear.

- Update(game.py) | Added 1 line

```
10     PIT: "Fell in a pit! Back to start!",
11 }
12 SPEED = 3
13 +GUARD_SPEED = 2
14
15 def generate_world():
16     grid = [[WALL]*COLS for _ in range(ROWS)]
```
- Update(game.py) | Added 7 lines, removed 4 lines

```
48     cr, ck = rooms[-1], rooms[-2]
49     grid[cr.centery][cr.centerx] = CHEST
50     grid[ck.centery][ck.centerx] = KEY
51 -     place_traps(grid, rooms, corridor)
51 +     guard_row = cr.centery - 1
52 +     guard_tiles = {(x, guard_row) for x in range(cr.left, cr.right)}
53 +     place_traps(grid, rooms, corridor | guard_tiles)
54
55     start = rooms[0] if rooms else None
54 -     return grid, start
56 +     patrol = ((cr.left*TILE+6, guard_row*TILE+6), ((cr.right-1)*TILE+6, guard_row*TILE+6))
57 +     return grid, start, patrol
58
56 -def place_traps(grid, rooms, corridor):
59 +def place_traps(grid, rooms, keep_clear):
60     spots = [(x, y) for room in rooms[1:]]
61         for y in range(room.top, room.bottom)
62         for x in range(room.left, room.right)
60 -         if grid[y][x] == FLOOR and (x, y) not in corridor]
63 +         if grid[y][x] == FLOOR and (x, y) not in keep_clear]
64     for _ in range(50):
65         traps = random.sample(spots, min(len(spots), random.randint(3, 6)))
66         for x, y in traps:
```
- Update(game.py) | Added 18 lines

```
118         if self.has_key:
119             pygame.draw.circle(screen, (220,220,60), (self.rect
```

```

        t.right-6, self.rect.top+6), 5)
120
121 +class Guard:
122 +    def __init__(self, a, b):
123 +        self.rect = pygame.Rect(a[0], a[1], 28, 28)
124 +        self.a, self.b = a, b
125 +        self.target = b
126 +        self.color = (40,180,90)
127
128 +    def update(self):
129 +        tx, ty = self.target
130 +        dx = max(-GUARD_SPEED, min(GUARD_SPEED, tx - self.rect
+.x))
131 +        dy = max(-GUARD_SPEED, min(GUARD_SPEED, ty - self.rect
+.y))
132 +        self.rect.move_ip(dx, dy)
133 +        if self.rect.topleft == self.target:
134 +            self.target = self.a if self.target == self.b else
+ self.b
135 +
136 +    def draw(self, screen):
137 +        pygame.draw.rect(screen, self.color, self.rect, border
+_radius=6)
138 +
139 +
140 WIDTH = COLS * TILE
141 HEIGHT = ROWS * TILE + 50
142 FPS = 60

```

- Update(game.py)

- └ Added 1 line, removed 1 line

```

152         self.reset()
153
154     def reset(self):
155 -        self.grid, start = generate_world()
155 +        self.grid, start, patrol = generate_world()
156     if start:
157         sx = start.x * TILE + 6
158         sy = start.y * TILE + 6

```

- Update(game.py)

- └ Added 1 line

```

160         sx, sy = TILE+6, TILE+6
161         self.start_pos = (sx, sy)
162         self.player = Player(sx, sy)
163 +        self.guard = Guard(*patrol)
164         self.won = False
165         self.status = "Find the KEY, then the CHEST!"
166

```

- Update(game.py)

- └ Added 3 lines

```

194         self.status = "You need the KEY first!"
195         elif cell in TRAP_MESSAGES:
196             self.send_to_start(TRAP_MESSAGES[cell])
197 +        self.guard.update()
198 +        if self.player.rect.colliderect(self.guard.rect):
199 +            self.send_to_start("Caught by the guard! Back to s
+tart!")
200
201     def draw(self):
202         self.screen.fill((30,25,40))

```

- Update(game.py)

- └ Added 1 line

```

209         pygame.draw.circle(self.screen, (255,240,6
0),(c*TILE+TILE//2, r*TILE+TILE//2),10)
210         elif cell == CHEST:
211             pygame.draw.rect(self.screen,(180,120,20),
rect.inflate(-12,-12),border_radius=4)

```

```

212 +         self.guard.draw(self.screen)
213         self.player.draw(self.screen)
214         hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
215         pygame.draw.rect(self.screen,(20,20,35),hud)

```

- Now testing: every tile the guard passes over is floor (not chest, trap or wall), it bounces between its endpoints, and getting caught keeps the key.

Ran 1 shell command (ctrl+o to expand)

- I added the guard to game.py. It patrols the row of the chest room directly above the chest. I tested it with a headless script but didn't play it in the game window:
 - Patrol: Across 2,000 generated maps, the guard only ever touched plain floor tiles (never a wall, trap, key or chest tile), turned around at both ends, and every map could still be won.
 - Catching: When the guard caught a player holding the key, the player went back to the start position, kept the key, and got the right message. Pressing R creates a new guard.

1. Guard speed (game.py:13 (game.py#L13))

GUARD_SPEED = 2

The guard moves 2 pixels per frame, a bit slower than the player's 3, so you can get past it.

2. Patrol lane in generate_world() (game.py:51-57 (game.py#L51-L57))

```

guard_row = cr.centery - 1
guard_tiles = {(x, guard_row) for x in range(cr.left, cr.right)}
place_traps(grid, rooms, corridor | guard_tiles)

start = rooms[0] if rooms else None
patrol = ((cr.left*TILE+6, guard_row*TILE+6), ((cr.right-1)*TILE+6,
guard_row*TILE+6))
return grid, start, patrol

```

- The lane: It's the row just above the chest, from the room's left edge to its right edge.
- Always floor: Rooms are at least 3 tiles tall, so this row is always inside the chest room, and room tiles are always floor.
- Never the chest: The chest is one row lower.
- Never the key: The key is always in a different room.
- No traps: The lane tiles are passed to place_traps together with the corridor tiles, so traps are never placed there.
- The two patrol points: They're the room's first and last tile in that row, plus the same 6px offset the player uses. That puts the 28px guard in the middle of the row, so it covers only lane tiles and can never reach a wall.
- Return value: generate_world() now returns the patrol points as a third value.

3. place_traps parameter renamed (game.py:59-63 (game.py#L59-L63))

```
def place_traps(grid, rooms, keep_clear):
```

```
    ...
```

```
        if grid[y][x] == FLOOR and (x, y) not in keep_clear]
```

Only the name changed. It now means "tiles that must stay free of traps" instead of only corridors.

4. Guard class (game.py:121-138 (game.py#L121-L138))

```
class Guard:
```

```
    def __init__(self, a, b):
```

```
        self.rect = pygame.Rect(a[0], a[1], 28, 28)
```

```
        self.a, self.b = a, b
```

```
        self.target = b
```

```
        self.color = (40,180,90)
```

```
    def update(self):
```

```
        tx, ty = self.target
```

```
        dx = max(-GUARD_SPEED, min(GUARD_SPEED, tx - self.rect.x))
```

```

dy = max(-GUARD_SPEED, min(GUARD_SPEED, ty - self.rect.y))
self.rect.move_ip(dx, dy)
if self.rect.topleft == self.target:
    self.target = self.a if self.target == self.b else self.b

```

```

def draw(self, screen):

```

```

    pygame.draw.rect(screen, self.color, self.rect, border_radius=6)

```

- Movement: Each frame the guard steps toward its current target point, by at most GUARD_SPEED pixels on each axis. That limit also means it can't overshoot the target.
- Turning around: When it reaches the target exactly, it switches to the other point.
- Separate from Player: It has no keyboard input and no grid collision. It doesn't need collision because the lane is always clear.
- Look: It's a green rounded square. The player is a blue circle, spikes are red, pits are black, and the key and chest are yellow and gold, so it's easy to tell apart.

5. Creating the guard in reset() (game.py:153 (game.py#L153), game.py:160 (game.py#L160))

```

self.grid, start, patrol = generate_world()

```

```

...
self.guard = Guard(*patrol)

```

reset() runs on R, so a restart makes a new map, new traps and a new guard.

6. Guard movement and catching the player in update() (game.py:193-195 (game.py#L193-L195))

```

self.guard.update()

```

```

if self.player.rect.colliderect(self.guard.rect):

```

```

    self.send_to_start("Caught by the guard! Back to start!")

```

- Collision: It checks whether the player's rectangle overlaps the guard's.
- Reset: It reuses send_to_start from the traps task, which moves the player back without touching has_key.
- After a win: update() returns early once you've won, so the guard stops moving.
- Start is safe: The start room is never the chest room, so the player can't get caught again right after being sent back.

7. Drawing the guard in draw() (game.py:208 (game.py#L208))

```

self.guard.draw(self.screen)

```

It's drawn before the player, so the player shows on top when they overlap.

Trade-off: The guard's row can never have traps, so the chest room has slightly fewer places for them. That's what guarantees the guard never sits on a trap.

> **Now add Task 3: a mini-map. Make minimal changes to the existing code and don't rewrite the file.**

Draw a small mini-map in a corner of the screen (for example the top-right), built from self.grid every frame so it updates in real time.

Use different colours for walls and floor, and give the key and chest tiles their own marker while they are still on the map. Once the key is picked up it should disappear from the mini-map.

Mark the player's current position with a bright dot or square, computed from the player's tile position.

Give the mini-map a border and a semi-transparent or dark background so it stays readable, and keep it from covering the HUD text.

Put it in a separate draw_minimap method and call it from draw.

Show me only the changed or new code, with a short explanation of each change.

- I'll add a small MINI config next to COLORS, a draw_minimap method, and one call in draw().

- Update(game.py)
 - └ Added 7 lines

```

90     PIT: (20,15,10),
91 }
92
93 +MINI = 6 # mini-map pixels per tile
94 +MINI_COLORS = {
95 +     FLOOR: (130,125,115),
96 +     KEY: (255,240,0),
97 +     CHEST: (255,110,0),
98 +}
99 +
100 class Player:
101     def __init__(self, x, y):
102         self.rect = pygame.Rect(x, y, 28, 28)

```

- Update(game.py)

```

└ Added 1 line
218             pygame.draw.rect(self.screen, (180,120,20),
    rect.inflate(-12,-12),border_radius=4)
219         self.guard.draw(self.screen)
220         self.player.draw(self.screen)
221 +         self.draw_minimap()
222         hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
223         pygame.draw.rect(self.screen, (20,20,35),hud)
224         st = self.font.render(self.status+" | R=Restart", True, (200,200,200))

```

- Update(game.py)

```

└ Added 16 lines
233             self.screen.blit(sub, (WIDTH//2-sub.get_width()//2,
    ROWS*TILE//2+20))
234             pygame.display.flip()
235
236 +     def draw_minimap(self):
237 +         mini = pygame.Surface((COLS*MINI, ROWS*MINI), pygame.S
    +RCALPHA)
238 +         mini.fill((0,0,0,170))
239 +         for r in range(ROWS):
240 +             for c in range(COLS):
241 +                 cell = self.grid[r][c]
242 +                 if cell != WALL:
243 +                     color = MINI_COLORS.get(cell, MINI_COLORS[
    +FLOOR])
244 +                     mini.fill(color, (c*MINI, r*MINI, MINI, MI
    +NI))
245 +         pc = self.player.rect.centerx // TILE
246 +         pr = self.player.rect.centery // TILE
247 +         mini.fill((255,255,255), (pc*MINI, pr*MINI, MINI, MINI
    +))
248 +         x, y = WIDTH - mini.get_width() - 10, 10
249 +         self.screen.blit(mini, (x, y))
250 +         pygame.draw.rect(self.screen, (230,230,230), (x-1, y-1
    +, mini.get_width()+2, mini.get_height()+2), 1)
251 +
252         def run(self):
253             running=True
254             while running:

```

- Let me render a frame headlessly and look at it, before and after picking up the key.

Read 2 files, ran 1 shell command (ctrl+o to expand)

- I added the mini-map to the top-right corner of game.py. I rendered two frames headlessly to check it:
 - Before: the key showed as a yellow square, the chest as orange, and the player as a white square.
 - After walking onto the key: the yellow square was gone and the white square had moved to the key's old tile.
 - HUD: the bottom status bar stayed clear.

1. Mini-map settings (game.py:92-97 (game.py#L92-L97))

```
MINI = 6 # mini-map pixels per tile
MINI_COLORS = {
    FLOOR: (130,125,115),
    KEY: (255,240,0),
    CHEST: (255,110,0),
}
```

- Size: Each tile is drawn 6px wide, so the whole 20x15 map is 120x90px.
- Separate colours: The mini-map has its own palette because the main key and chest colours look too alike at 6px, and a bright yellow key doesn't stand out on beige floor. The mini-map uses grey floor, a bright yellow key and an orange chest instead.

2. draw_minimap() method (game.py:238-253 (game.py#L238-L253))

```
def draw_minimap(self):
    mini = pygame.Surface((COLS*MINI, ROWS*MINI), pygame.SRCALPHA)
    mini.fill((0,0,0,170))
    for r in range(ROWS):
        for c in range(COLS):
            cell = self.grid[r][c]
            if cell != WALL:
                color = MINI_COLORS.get(cell, MINI_COLORS[FLOOR])
                mini.fill(color, (c*MINI, r*MINI, MINI, MINI))
    pc = self.player.rect.centerx // TILE
    pr = self.player.rect.centery // TILE
    mini.fill((255,255,255), (pc*MINI, pr*MINI, MINI, MINI))
    x, y = WIDTH - mini.get_width() - 10, 10
    self.screen.blit(mini, (x, y))
    pygame.draw.rect(self.screen, (230,230,230), (x-1, y-1,
mini.get_width()+2, mini.get_height()+2), 1)
```

- Background and walls: The mini-map starts as a see-through black layer (alpha 170). Wall tiles are simply left as that dark background, which makes them clearly different from floor.
- Updates every frame: The mini-map is rebuilt from self.grid each frame. When you pick up the key, update() turns that tile back into FLOOR, so the key marker disappears from the mini-map automatically.
- Traps are hidden: Any tile without its own colour (spikes and pits) is drawn as floor, so the mini-map shows the layout but doesn't show where the traps are. If you'd rather it did, add them to MINI_COLORS. The pit needs a colour other than black, or it will look like a wall.
- Player marker: It's a white square on the tile under the player's centre, the same tile that triggers pickups and traps.
- Placement: It sits 10px from the top-right corner with a 1px light border. The HUD is below the play area, at $y \geq 600$, so the mini-map can never cover it.

3. Calling it from draw() (game.py:222 (game.py#L222))

```
self.draw_minimap()
```

It's drawn after the map, guard and player, so it stays on top of them. It's drawn before the win overlay, so the overlay dims it along with everything else.

The guard isn't shown on the mini-map, since you didn't ask for that. To add it, fill one more cell from self.guard.rect, the same way as the player marker.

> Now add Task 4: an inventory UI. Make minimal changes to the existing code and don't rewrite the file.

Add a HUD inventory slot in the bottom bar (or another clear spot that doesn't overlap the status text or mini-map). It should be a square box with a border and a small "INV" or "Item" label. At the start of the game the slot is empty. Once the player collects the key, it shows a key icon, drawn with pygame shapes (no image files or new dependencies). The slot should read self.player.has_key each frame so it updates immediately on pickup, and goes back to empty after R.

Put it in a separate `draw_inventory` method and call it from `draw`.
Show me only the changed or new code, with a short explanation of each change.

- The bottom bar is 50px tall and the status text starts at the left, so I'll put the slot at the far right of the bar. The mini-map is at the top, so the two can't overlap.

- Update(game.py)

```
└─ Added 1 line
156         self.clock = pygame.time.Clock()
157         self.font = pygame.font.SysFont("monospace", 24)
158         self.big_font = pygame.font.SysFont("monospace", 40, bold=True)
159 +       self.small_font = pygame.font.SysFont("monospace", 14,
+ bold=True)
160         self.reset()
161
162     def reset(self):
```

- Update(game.py)

```
└─ Added 1 line
224         pygame.draw.rect(self.screen, (20,20,35), hud)
225         st = self.font.render(self.status+" | R=Restart", True,
ue, (200,200,200))
226         self.screen.blit(st, (8,ROWS*TILE+13))
227 +       self.draw_inventory()
228         if self.won:
229             ov=pygame.Surface((WIDTH,ROWS*TILE),pygame.SRCALPHA)
A)
230             ov.fill((0,0,0,140))
```

- Update(game.py)

```
└─ Added 14 lines
235         self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,
ROWS*TILE//2+20))
236         pygame.display.flip()
237
238 +     def draw_inventory(self):
239 +         slot = pygame.Rect(WIDTH-46, ROWS*TILE+7, 36, 36)
240 +         label = self.small_font.render("INV", True, (160,160,1
+60))
241 +         self.screen.blit(label, (slot.x - label.get_width() -
+6, slot.centery - label.get_height()//2))
242 +         pygame.draw.rect(self.screen, (45,45,65), slot, border
+_radius=4)
243 +         pygame.draw.rect(self.screen, (200,200,200), slot, 2,
+border_radius=4)
244 +         if self.player.has_key:
245 +             color = (255,240,60)
246 +             cy = slot.centery
247 +             pygame.draw.circle(self.screen, color, (slot.x+11,
+ cy), 6, 3)
248 +             pygame.draw.line(self.screen, color, (slot.x+16, c
+y), (slot.right-6, cy), 3)
249 +             pygame.draw.line(self.screen, color, (slot.right-8
+, cy), (slot.right-8, cy+6), 3)
250 +             pygame.draw.line(self.screen, color, (slot.right-1
+3, cy), (slot.right-13, cy+5), 3)
251 +
252         def draw_minimap(self):
253             mini = pygame.Surface((COLS*MINI, ROWS*MINI), pygame.S
RCALPHA)
254             mini.fill((0,0,0,170))
```

- Now checking that the longest status message doesn't run into the label, and rendering the empty, key and after-R states:

Read 2 files, ran 1 shell command (ctrl+o to expand)

- I added the inventory slot to the far right of the bottom bar in game.py. I checked it with rendered frames rather than by playing:
 - Empty at the start: the slot shows only the dark box and its border.
 - Key: once has_key is true, a yellow key icon appears in the box.
 - After R: has_key goes back to False, so the slot is empty again.
 - No overlap: the longest status message ("Caught by the guard! Back to start! | R=Restart") ends at x=694 and the "INV" label starts at x=723. That leaves about 29px between them. The mini-map is at the top of the screen, so the two can't touch.

1. Small font for the label (game.py:147 (game.py#L147))

```
self.small_font = pygame.font.SysFont("monospace", 14, bold=True)
```

It's created once in `__init__` with the other fonts, rather than every frame.

2. Calling it from draw() (game.py:233 (game.py#L233))

```
self.draw_inventory()
```

It's called right after the status text is drawn in the bottom bar. The win overlay only darkens the play area, so the slot stays visible after you win.

3. draw_inventory() method (game.py:244-256 (game.py#L244-L256))

- ```
def draw_inventory(self):
 slot = pygame.Rect(WIDTH-46, ROWS*TILE+7, 36, 36)
 label = self.small_font.render("INV", True, (160,160,160))
 self.screen.blit(label, (slot.x - label.get_width() - 6, slot.centery
- label.get_height()//2))
 pygame.draw.rect(self.screen, (45,45,65), slot, border_radius=4)
 pygame.draw.rect(self.screen, (200,200,200), slot, 2, border_radius=4)
 if self.player.has_key:
 color = (255,240,60)
 cy = slot.centery
 pygame.draw.circle(self.screen, color, (slot.x+11, cy), 6, 3)
 pygame.draw.line(self.screen, color, (slot.x+16, cy),
(slot.right-6, cy), 3)
 pygame.draw.line(self.screen, color, (slot.right-8, cy),
(slot.right-8, cy+6), 3)
 pygame.draw.line(self.screen, color, (slot.right-13, cy),
(slot.right-13, cy+5), 3)
```
- The box: It's 36×36, sits 10px from the right edge, and is centred vertically in the 50px bar. It has a dark fill and a 2px light border.
  - The label: "INV" is drawn to the left of the box and centred on it vertically.
  - Updates immediately: The method reads self.player.has\_key every frame and stores nothing of its own. So the icon appears on the same frame you pick up the key.
  - Clears on R: reset() creates a new Player with has\_key = False, so the slot empties on restart. Traps and the guard keep the key, so the icon stays after those.
  - The key icon: It uses only pygame shapes: a ring for the bow, a line for the shaft and two short lines for the teeth. It's the same yellow as the key on the map.

The "INV" label is small at size 14. If you want it easier to read, raise the size in the small\_font line. At size 16 or 18 it still clears the status text.